Qualifying Exam

November 7, 2025

Sam Buxbaum

Motivation

Secure computation has taken great leaps in recent years...

but it's still impractical in many scenarios.

Goal: Fill in the gaps that prevent the use of MPC in practice

Scaling Up MPC

1. More sophisticated functions

- Oblivious shuffling
- Improving the expressivity of MPC

2. Directly incorporating more parties

- Significant practical challenges
- Distribute trust more widely

Oblivious Shuffle Preprocessing

Oblivious Shuffling

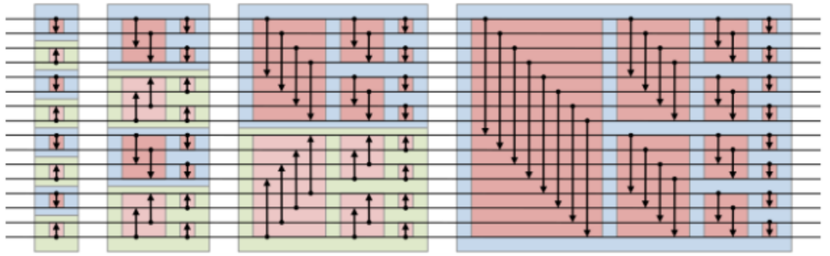
Two parties, P_0 and P_1, want to shuffle a secret-shared vector $[\mathbf{x}]$ by a permutation π.

- Neither party should learn anything about π
- The output is a secret sharing $[\pi(\mathbf{x})]$

Example: Oblivious Sorting

Oblivious sorting networks require $O(n \log^2 n)$ comparison gates.

- We want to match the plaintext $O(n \log n)$ algorithms



The shuffle-then-sort paradigm.

- Obliviously shuffle, then use a data-dependent sorting algorithm
- The revealed permutation is uncorrelated with the input
- Achieves $O(n \log n)$ secure comparisons

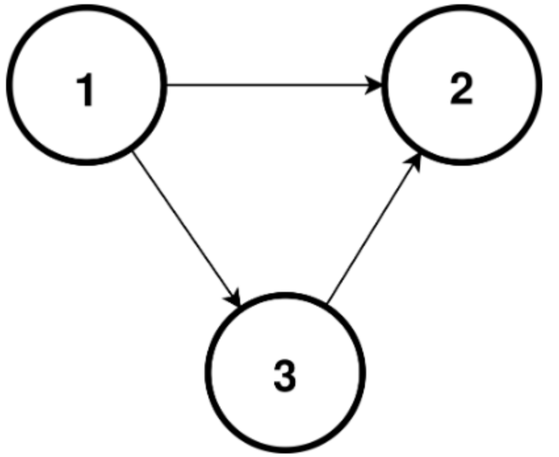
Example: Graph Algorithms

Not easy to efficiently express a graph obliviously.

- Lots of data-dependent operations

The GraphSC paradigm.

- Encode the graph as a single list
- The list has two orderings
- Two important operations, scatter and gather
- Each operation is efficiently only in one of the orderings
- Shuffling allows us to convert between the orderings



Broader Applications

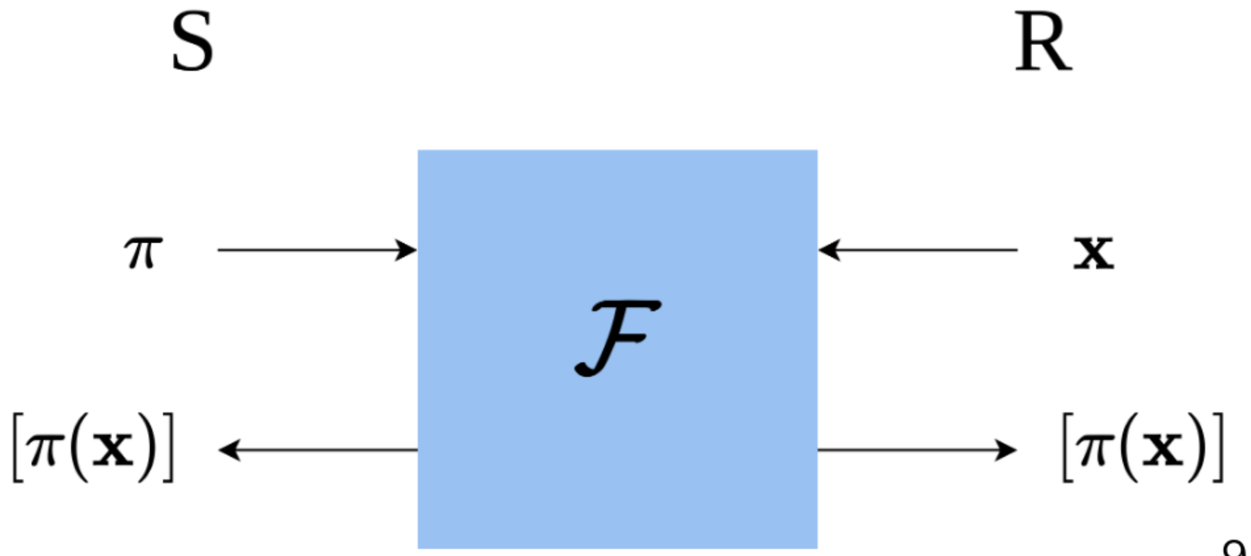
Combine aspects of the circuit model and RAM model of computation.

- A bridge between the two worlds
- Expressivity makes it central to complex workloads

How do we do it?

Break the Problem in Half

- Let $\pi = \pi_1 \circ \pi_0$ be a composition of random permutations
  - P_0 knows π_0
  - P_1 knows π_1
  - Neither party knows anything about π
- One party acts as a sender, the other as a receiver
- Known as "Permute+Share"



Preprocessing

Permute+Share can be broken into preprocessing and online phases.

- The preprocessing phase shuffles pseudorandom values
- The online phase derandomizes in a single message

Preprocessing produces a *permutation correlation*.

$$(\pi, C), (A, B)$$

$$\pi(A) = B + C$$

Online Derandomization Protocol

$\text{S: } (\pi, C)$ $\qquad\qquad$ $\text{R: } (\mathbf{x}, (A, B))$

$$\mathbf{x} - A \longleftarrow$$

$\pi(\mathbf{x} - A) + C$ $\qquad\qquad$ B

$$\begin{aligned}
\text{output} &= \pi(\mathbf{x} - A) + C + B \\
&= \pi(\mathbf{x} - A) + \pi(A) \\
&= \pi(\mathbf{x}) - \pi(A) + \pi(A) \\
&= \pi(\mathbf{x})
\end{aligned}$$

Formally Defining the Problem

Functionality $\mathcal{F}_{\text{GenPerm}}$

PUBLIC PARAMETERS: Number of elements n and bitwidth ℓ.
INPUT: None
OUTPUT: The functionality samples a uniformly random permutation $\pi \leftarrow S_n$ and uniformly random vectors $A, B, C \in \{0,1\}^{n \times \ell}$, such that $\pi(A) = B + C$. $\mathcal{F}_{\text{GenPerm}}$ outputs (π, C) to the sender and (A, B) to the receiver.

Three parameters we care about:

- n, the number of elements in a permutation correlation
- ℓ, the bitwidth of each element of A, B, and C in a single permutation correlation
- p, the total number of permutation correlations generated

State of the Art

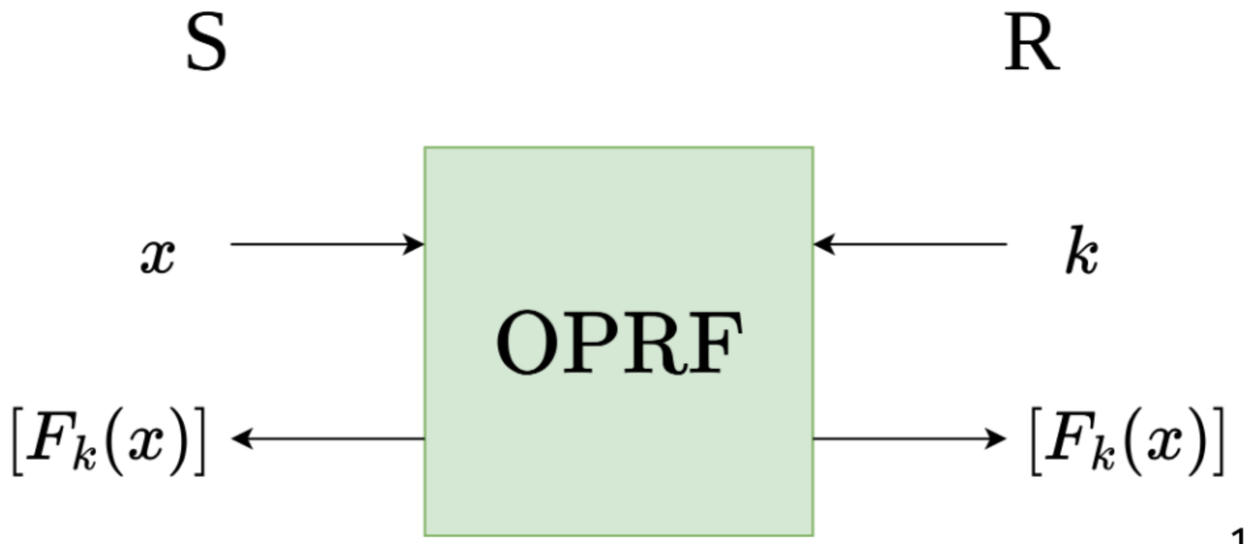
Peceny, Raghuraman, Rindal, and Shah (PKC 2025)

- Two constructions
- Both based on oblivious pseudorandom functions
- Both with linear communication in n
- One with linear communication in ℓ
- One with sublinear communication in ℓ

Oblivious Pseudorandom Functions (OPRFs)

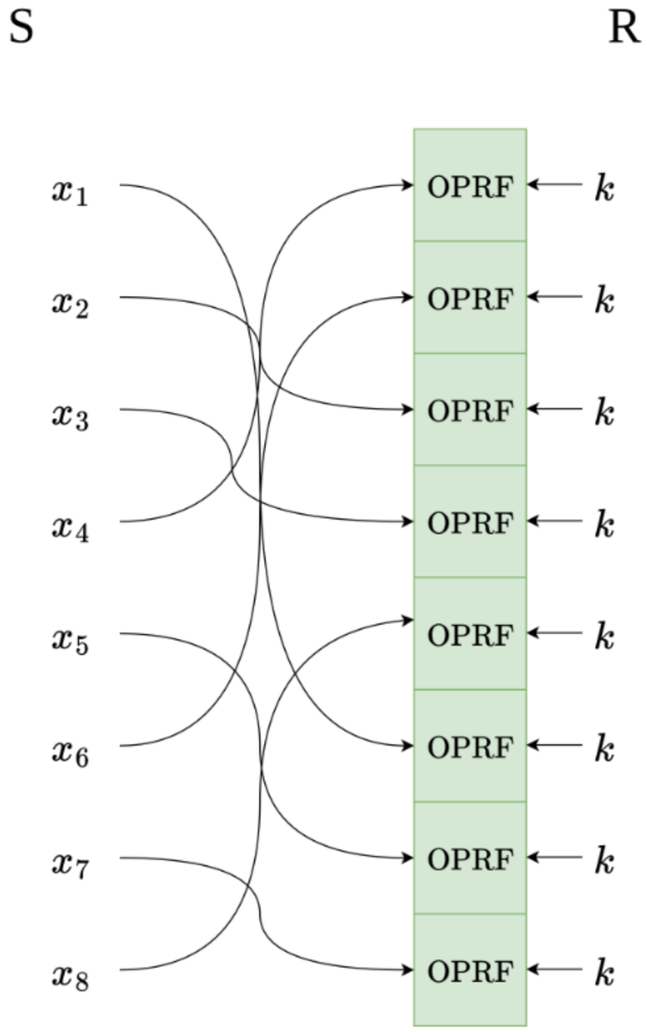
A core building block of each construction is an OPRF with secret-shared output.

- Sender provides an input x
- Receiver provides a key k
- Instantiated with the alternating-moduli family of PRFs
- Outputs a secret sharing



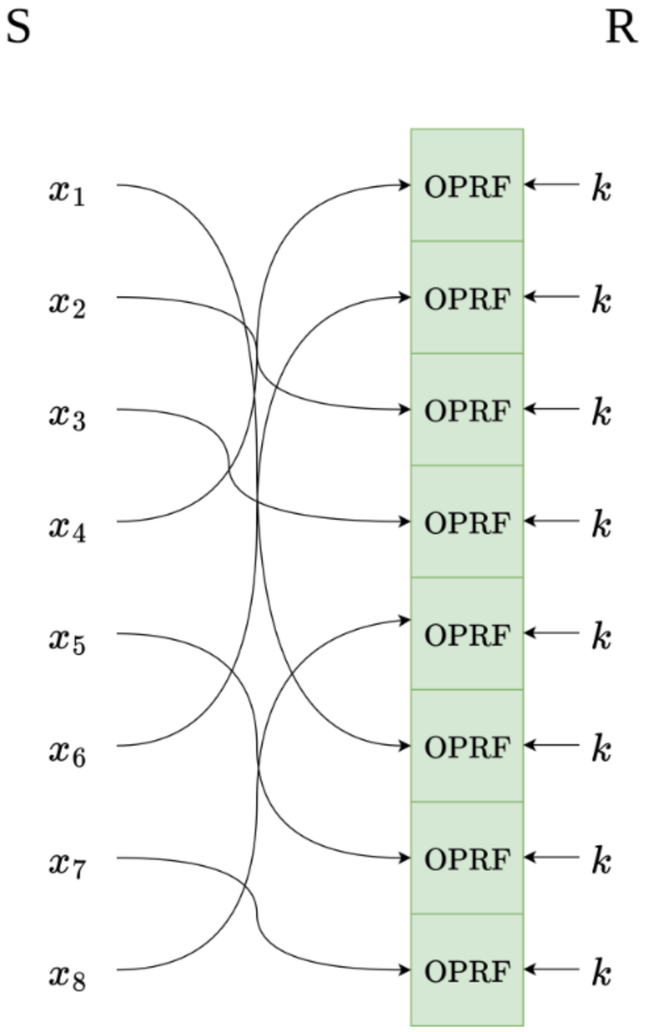
Primary Construction

- Parties agree on a random vector $\mathbf{x}$ by computing $x_i = H(i)$
- S samples $\pi \leftarrow S_n$
- S permutes $\mathbf{x}$ by π to set OPRF inputs
- R samples a random PRF key k
- R sets $A_i = F_k(x_i)$ in plaintext
- Parties set B and C as the OPRF outputs



Analysis of the Primary Construction

- n calls to the OPRF
- $O(n\ell)$ total communication
- Most efficient construction to date in practice
- ≈ 4s per million elements



Sublinear Protocol

Treat the output of the primary construction as a vector of seeds to pseudorandom correlation generators.

- Let t be some sparsity parameter
- Each $\log(2\ell/t)$ bits encodes a unit vector of length $2\ell/t$
- Compute each unit vector using a distributed point function (DPF)
- Concatenate t unit vectors to obtain a t-sparse vector of length 2ℓ
- Compress into a pseudorandom vector of length ℓ
  - Reduction to syndrome decoding assumption

Analysis of the Sublinear Protocol

- $O(n \log \ell)$ total communication
- Efficient only for very long ℓ
- Crossover point is $\ell \approx 4000$
- Application: graph algorithms

Open Problems

Goal: Extend the line of work on PCGs into the domain of oblivious shuffling.

We want to do to oblivious shuffling what pseudorandom correlation generators did to oblivious transfer.

Concrete technical problems:

- Sublinearity in ℓ (in practice)
- Sublinearity in p
- Sublinearity in n

Sublinearity in ℓ

Protocols exist for generating permutation correlations with sublinear communication in ℓ.

- No implementation of the Peceny et al. construction (until now!)
- Improve parameters to get practical efficiency

Ongoing work: graph algorithms

- Requires thousands of bits of permutation correlations
- Existing constructions can be practical

Sublinearity in p

Can we "chop up" a correlation with large ℓ into many correlations with small ℓ?

- Very challenging
- Can define a *stateful rerandomization* ideal functionality that captures the intended behavior

Application: relational analytics

Sublinearity in n

The holy grail of this research direction.

- Make shuffling an efficient building block for *any* workload

Preliminary ideas:

- Generate many small correlations, then "stitch them together"
- Construct from a protocol with sublinearity in p
- Generate small correlations directly by materializing the permutation matrix

Scaling with Many Parties

Motivation

Challenge: Suppose a secure computation involves thousands of parties.

N Party Computation

- Large body of theoretical work
- Prohibitive practical challenges

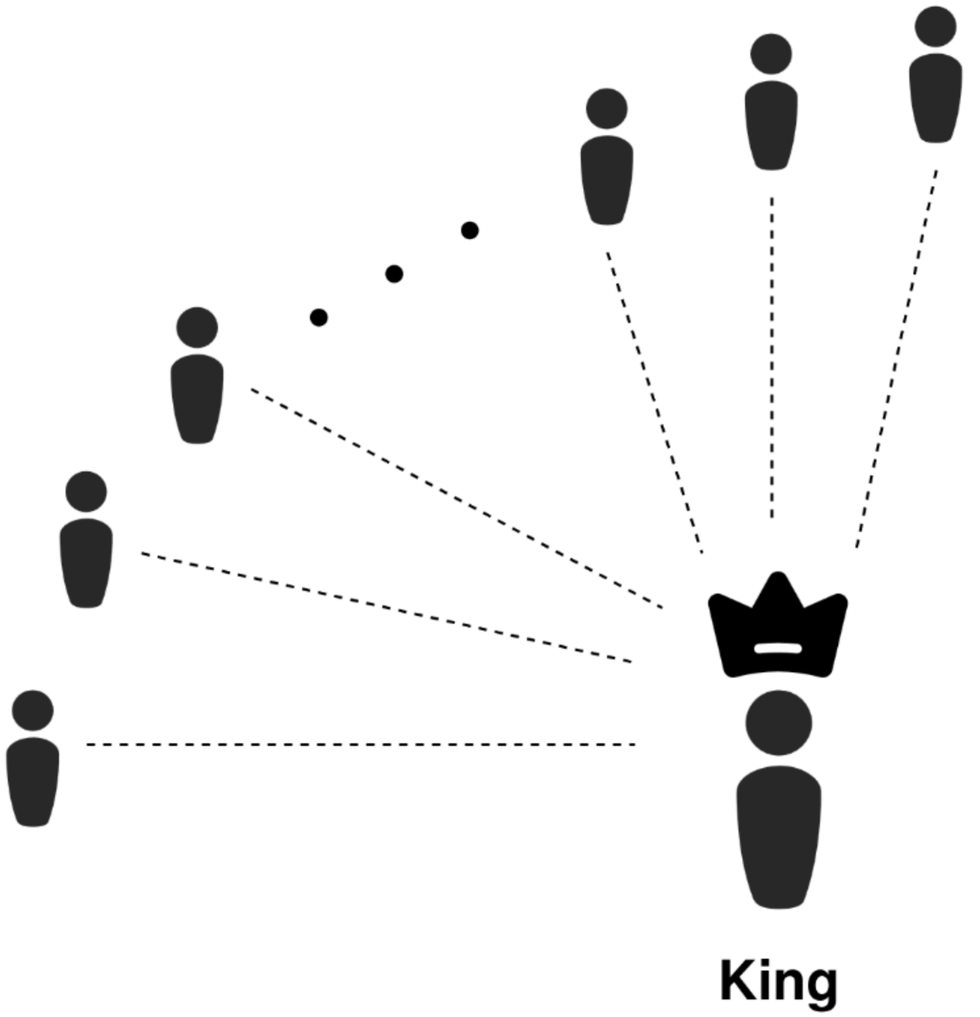
The Outsourced Setting

- Practical and efficient
- Requires stronger trust assumptions

Goal: Scale *practical* MPC to thousands of parties.

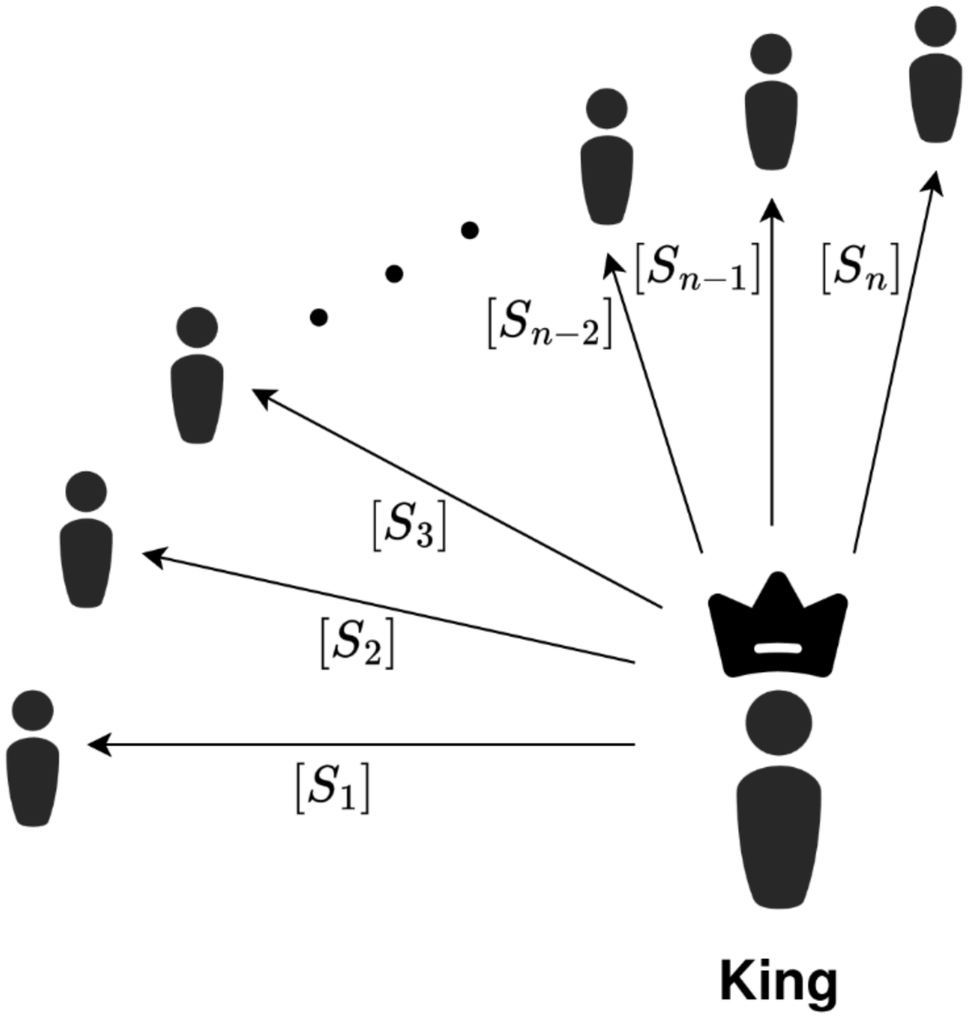
A Star Topology

- Damgard-Nielsen 2007
- Central untrusted "king" party
- $O(n)$ communication per gate
- Based on Shamir secret sharing
- Requires an honest majority



DN07 Multiplication

- $[x]$ denotes a degree-t Shamir sharing
- $\langle x \rangle$ denotes a degree-$2t$ Shamir sharing
- Local multiplication converts t sharing to $2t$ sharing
- We need to perform a degree reduction
- Multiplication consumes a double sharing $([r], \langle r \rangle)$



Extending to More Complex Functions

Liu et al. (Usenix Security 2024)

- Based on the DN07 protocol
- Uses the properties of Mersenne prime fields
- Bitwise operations on arithmetic secret shares
- Targets machine learning inference
- Optimizes for round complexity
- Shows results with up to 63 parties

Liu et al. Truncation Protocol

- Division by a public power of two

Almost all truncation protocols either

- Require an arithmetic-to-binary conversion
- Require "slack bits" to prevent a "catastrophic" error

The Liu et al. protocol is both constant-round and requires only 1 bit of slack.

- In a Mersenne prime field, the catastrophic error is predictable and can be corrected for

The paper simultaneously minimizes total work and makes base operations constant-round.

Liu et al. in Practice

LAN Setting

- Bottlenecked by local computation on the king party
- King needs to perform polynomial interpolation for each gate

WAN Setting

- Latency constrained
- Uses 5% of available bandwidth

Scaling the King Party

Proposal: Lean into the king as the bottleneck.

- Powerful central server, weak ephemeral clients
- One logical king, multiple servers
- GPU acceleration

Tolerating Dropouts

Problem: Slowest connection becomes the bottleneck.

Proposal: Embrace the threshold scheme to tolerate dropouts.

- Subset of Fluid MPC
- Semi-honest security is straightforward
- Malicious security is more complex

Thank You!